

Digital twin project presentation

Project requirements

- GOAL: To build a script that allows to generate the **Digital Twin of an SDN network**
- Exploit **RYU Northbound RestAPI** to retrieve the topology- and traffic-related information
- The procedure should be **completely automated**
- **Runtime changes to the Physical Twin are reproduced automatically** to the Digital Twin

Additional features

- Interactive **live CLI** to interrogate the digital twin status
- Basic “**what-if**” **calculations** on the digital twin to predict changes before applying them on the real network

High-level system architecture

rest_client.py	Handles all low-level HTTP communication with Ryu's REST API. Provides methods like <code>get_switches()</code>, <code>get_links()</code>, <code>get_hosts()</code>, <code>get_port_stats()</code>, <code>get_flow_stats()</code>, <code>get_port_description()</code>
dt_model.py	Implements the graph data structure (the Digital Twin) using <code>networkx.MultiGraph</code>. It stores switches, hosts, edges, and all attributes. Also contains the simulation logic (<code>clone()</code>, <code>add_hypothetical_flow()</code>, <code>simulate_impact()</code>)

High-level system architecture

dt_sync.py	Contains the DigitalTwinSync class that runs a periodic polling loop. It calls the REST client, updates the model, and logs changes.
dt_cli.py	Provides an interactive Command Line Interface (CLI) using Python's cmd module. Users can query the twin and run what-if simulations.

The physical environment

Mininet

The main topologies that I tested are

- single,2 ; single,3
- linear,2 ; linear,3
- tree, depth=2, fanout=2
- I also briefly tested a custom non fully-connected topology

The physical environment

Ryu

```
ryu-manager --verbose ryu.app.rest_topology  
ryu.app.ofctl_rest ryu.app.simple_switch_1
```

rest_topology – exposes topology data (switches, links, hosts) under /v1.0/topology/.

ofctl_rest – exposes OpenFlow statistics (port stats, flow stats) under /stats/.

simple_switch_13 – implements MAC-learning forwarding (L2 switching). This creates flows dynamically as traffic flows.

networkx.multigraph

- Each switch-switch link that we get through the API appear twice as two bidirectional links, but I collapse them as one non directional link.

```
interface1 = (src_dpid, src_port)
interface2 = (dst_dpid, dst_port)
non_directional_link_key = tuple(sorted([interface1, interface2]))
```


Node types and attributes

<i>Node type</i>	<i>Node ID</i>	<i>Attributes</i>
<u>Switch</u>	DPID	ports (list of port dicts) port_stats (dict per port) flows (list of real flow entries) hypothetical_flows (list, only in clones)
<u>Host</u>	MAC address	ipv4 (list) ipv6 (list) connected_to (switch DPID) connected_port (int of the switch port)

Edge types and attributes

<i>Edge type</i>	<i>Edge key</i>	<i>Attributes</i>
<u>switch-switch</u>	((dpid1, port1), (dpid2, port2))	state (1 for UP, 0 for DOWN) no separate src_port/dst_port because ports are in the key
<u>host-switch</u>	(host_mac, switch_dpid, "host_switch")	switch_port (int) state ("up" or "down")

Synchronization

Updating the digital twin works with a **pull-based** approach.

The default rate is every **2 seconds**, but can be modified

The fetch and update cycle

- **Fetch Topology** (*switches, links, hosts*)
 - Update graph structure (*update_switches, update_links, update_hosts*)
- For each switch, **fetch Port Stats and Flow Stats**
 - Update node attributes (*update_port_stats, update_flow_stats*)
- For each switch, **fetch Port Description**
 - Update link states (*update_host_link_states, update_switch_link_states*)

Diff detection

The system keeps a **snapshot of the previous graph state (as a dict)**.

On each new cycle, it compares the current state with the previous one and prints a clean log of exactly what changed. This is useful so that the user can have immediate feedback.

- e.g., *[CHANGE] Node added: ..., [STATE] Link ... is now DOWN.*

Features on the digital twin

- Interactive CLI
- What-if simulation

Interactive CLI

The available commands are:

- help
- summary
- hosts
- switches
- links
- flows
- whatif
- save
- quit

What-if simulation

- A little engine that aims at previewing some topology changes
- It does so by:
 - applying a hypothetical flow to one specific switch
 - Returning back to the user how the other real flows would be impacted

What-if simulation

Major features:

- Packet DROP:
 - drops packets that match the hypothetical flow
- Packet OUTPUT:
 - Changes the output port of the packets that match the hypothetical flow

DEMO TIME!

:)